

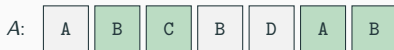
Dynamic Programming, Part 2

LCS, Knapsack, Weighted Interval Scheduling

Longest Common Subsequence

Subsequence, not Substring

A **subsequence** deletes characters without reordering the rest. A **substring** must be contiguous.



BCAB is a subsequence of ABCBDAB, but not a substring.

The Problem

Given $A[1 \dots n]$ and $B[1 \dots m]$, find the longest sequence that is a subsequence of **both**.

A:

A	B	C	B	D	A	B
---	---	---	---	---	---	---

B:

B	D	C	A	B
---	---	---	---	---

BCAB has length 4. So does BDAB.

The LCS need not be unique.

The Table

Definition

$LCS[i, j]$ = length of the LCS of $A[1 \dots i]$ and $B[1 \dots j]$.

Look at the last characters. Two cases.



match: extend



mismatch: drop one

The Recurrence

- $A[i] = B[j]$: match them and extend, giving $LCS[i - 1, j - 1] + 1$.

The Recurrence

- $A[i] = B[j]$: match them and extend, giving $LCS[i - 1, j - 1] + 1$.
- $A[i] \neq B[j]$: the LCS cannot end with both, so drop one of them and take the better option.

The Recurrence

- $A[i] = B[j]$: match them and extend, giving $LCS[i - 1, j - 1] + 1$.
- $A[i] \neq B[j]$: the LCS cannot end with both, so drop one of them and take the better option.

$$LCS[i, j] = \begin{cases} LCS[i - 1, j - 1] + 1 & \text{if } A[i] = B[j], \\ \max(LCS[i - 1, j], LCS[i, j - 1]) & \text{otherwise.} \end{cases}$$

Base cases: $LCS[i, 0] = 0$ and $LCS[0, j] = 0$.

A Worked Table

$$A = \text{ABCB}, \quad B = \text{BCB}.$$

		B	C	B
A	0	0	0	0
	0	0	0	0
B	0	1	1	1
C	0	1	2	2
B	0	1	2	3

$LCS[4, 3] = 3$, achieved by BCB.

Implementation: Filling the Table

Row by row, left to right.

```
function lcs(A::String, B::String)
    n, m = length(A), length(B)
    L = OffsetArray(zeros{Int, 2}, n + 1, m + 1), 0:n, 0:m

    for i in 1:n
        for j in 1:m
            if A[i] == B[j]
                L[i, j] = L[i - 1, j - 1] + 1
            else
                L[i, j] = max(L[i - 1, j], L[i, j - 1])
            end
        end
    end

    # continued on the next slide
```

Implementation: Recovering the String

Walk back from (n, m) , prepending a character on every match.

```
# ... still inside lcs(A, B)

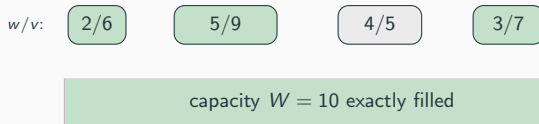
lcs_str = ""
i, j = n, m
while i > 0 && j > 0
    if A[i] == B[j]
        lcs_str = string(A[i]) * lcs_str
        i -= 1; j -= 1
    elseif L[i - 1, j] > L[i, j - 1]
        i -= 1
    else
        j -= 1
    end
end

return L[n, m], lcs_str
end
```

0/1 Knapsack: Maximize Value

The Problem

n items, item i has weight $w[i]$ and value $v[i]$. Knapsack capacity W . Each item is taken whole or not at all.



Items 1, 2, 4: weight $2 + 5 + 3 = 10$, value $6 + 9 + 7 = 22$.

Greedy Fails

Sorting by value-to-weight ratio is tempting, but wrong.

$W = 40$, items $(20, 100)$, $(20, 100)$, $(25, 150)$.

greedy: (25, 150) ratio 6, then nothing fits: 150

optimal: (20, 100) (20, 100) ratio 5 each: 200

The Table

Definition

$K[i, c]$ = maximum value using items $1 \dots i$ with capacity c .

For item i at capacity c :

- **Skip it:** capacity unchanged, value $K[i - 1, c]$.

The Table

Definition

$K[i, c]$ = maximum value using items $1 \dots i$ with capacity c .

For item i at capacity c :

- **Skip it:** capacity unchanged, value $K[i - 1, c]$.
- **Take it:** only if $w[i] \leq c$; spend $w[i]$, gain $v[i]$, giving $K[i - 1, c - w[i]] + v[i]$.

The Table

Definition

$K[i, c]$ = maximum value using items $1 \dots i$ with capacity c .

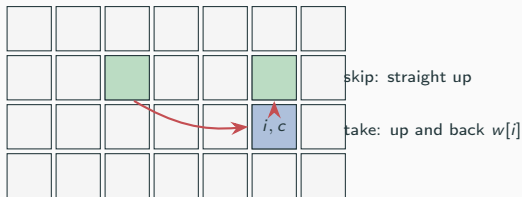
For item i at capacity c :

- **Skip it:** capacity unchanged, value $K[i - 1, c]$.
- **Take it:** only if $w[i] \leq c$; spend $w[i]$, gain $v[i]$, giving $K[i - 1, c - w[i]] + v[i]$.

$$K[i, c] = \begin{cases} K[i - 1, c] & \text{if } w[i] > c, \\ \max(K[i - 1, c], K[i - 1, c - w[i]] + v[i]) & \text{otherwise.} \end{cases}$$

Filling the Table

Base cases: $K[0, c] = 0$ and $K[i, 0] = 0$.



Row i reads only row $i - 1$, so fill row by row. Answer at $K[n, W]$.

Running time $O(nW)$.

Implementation

```
function knapsack(weights, values, W)
    n = length(weights)
    K = OffsetArray(zeros{Int, n + 1, W + 1}, 0:n, 0:W)

    for i in 1:n
        for c in 0:W
            K[i, c] = K[i - 1, c] # skip item i
            if weights[i] <= c
                K[i, c] = max(K[i, c],
                    K[i - 1, c - weights[i]] + values[i]) # take item i
            end
        end
    end

    selected_items = [] # trace back
    c = W
    for i in n:-1:1
        if K[i, c] != K[i - 1, c] # value changed, so item i was taken
            push!(selected_items, i)
            c -= weights[i]
        end
    end

    return K[n, W], selected_items
end
```

0/1 Knapsack: Minimize Weight

Flip the Table Around

Index by **value** instead of capacity. Let $V_{\max} = \sum_{i=1}^n v[i]$.

Definition

$M[i, u]$ = minimum weight to achieve **exactly** value u using items $1 \dots i$, or $+\infty$ if impossible.

This version pays off later, when we approximate knapsack in time independent of W .

The Recurrence

For item i at target value u :

- **Skip it:** $M[i - 1, u]$.

The Recurrence

For item i at target value u :

- **Skip it:** $M[i - 1, u]$.
- **Take it:** only if $v[i] \leq u$; add weight $w[i]$ and ask the rest for value $u - v[i]$, giving $M[i - 1, u - v[i]] + w[i]$.

The Recurrence

For item i at target value u :

- **Skip it:** $M[i - 1, u]$.
- **Take it:** only if $v[i] \leq u$; add weight $w[i]$ and ask the rest for value $u - v[i]$, giving $M[i - 1, u - v[i]] + w[i]$.

$$M[i, u] = \begin{cases} M[i - 1, u] & \text{if } v[i] > u, \\ \min (M[i - 1, u], M[i - 1, u - v[i]] + w[i]) & \text{otherwise.} \end{cases}$$

Base cases: $M[0, 0] = 0$, and $M[0, u] = +\infty$ for $u > 0$.

Reading Off the Answer

The table gives minimum weight per value. We want the best value that **fits**.

u :	0	1	2	3	4	5	6
$M[n, u]$:	0	3	5	8	9	12	14

$W = 10$

Scan u from 0 to $V_{\max}$ and keep the largest with $M[n, u] \leq W$.

Here the answer is $u = 4$.

Running time $O(n \cdot V_{\max})$.

Implementation

```
function knapsack_weight(weights, values, W)
    n = length(weights)
    V_max = sum(values)
    INF = div(typemax{Int}, 2) # halved to avoid overflow
    M = OffsetArray(fill(INF, n + 1, V_max + 1), 0:n, 0:V_max)
    M[0, 0] = 0

    for i in 1:n
        for u in 0:V_max
            M[i, u] = M[i - 1, u] # skip item i
            if values[i] <= u && M[i - 1, u - values[i]] < INF
                M[i, u] = min(M[i, u],
                    M[i - 1, u - values[i]] + weights[i]) # take item i
            end
        end
    end

    best = 0 # largest value that fits
    for u in 0:V_max
        if M[n, u] <= W
            best = u
        end
    end
end
```

Exercise

1. Modify `knapsack_weight` to also return *which* items are selected, like the traceback in the value-based version.
2. Both versions take each item at most once. What changes if an item may be taken **any number of times**?

The second one is the *unbounded* knapsack problem.

Solution 1

Same trick as before: compare against the skip case.

Walk back from (n, u^*) where u^* is the answer. At row i :

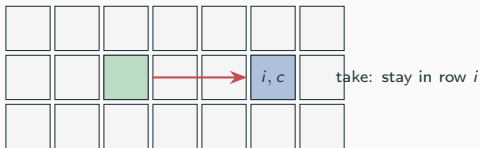
- If $M[i, u] = M[i - 1, u]$, item i was skipped. Move to $(i - 1, u)$.
- Otherwise item i was taken. Record it and move to $(i - 1, u - v[i])$.

The walk touches one cell per row, so it costs $O(n)$.

Solution 2

If item i can be reused, then after taking it we may take it **again**. So the second term looks at row i , not row $i - 1$:

$$K[i, c] = \max \left(K[i - 1, c], \underbrace{K[i, c - w[i]]}_{\text{row } i} + v[i] \right)$$

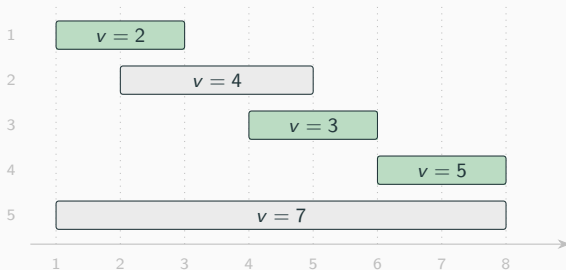


One dimension collapses: $K[c] = \max_i (K[c - w[i]] + v[i])$.

Weighted Interval Scheduling

The Problem

n jobs, job i has start $s[i]$, finish $f[i]$, and weight $v[i] > 0$. Two jobs are **compatible** if they do not overlap.



Jobs 1, 3, 4 are compatible with total weight **10**, beating job 5 alone at 7.

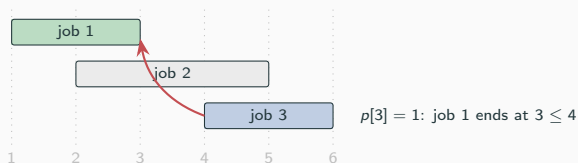
Preprocessing

Sort the jobs by finish time: $f[1] \leq f[2] \leq \dots \leq f[n]$.

Definition

$p[i]$ = the largest $j < i$ with $f[j] \leq s[i]$, or 0 if none exists.

In words: the **latest job that finishes before job i starts**.



Computable in $O(n \log n)$ by binary search after sorting.

The Recurrence

For job i :

- **Skip it:** the best among the first $i - 1$ jobs, $dp[i - 1]$.

The Recurrence

For job i :

- **Skip it:** the best among the first $i - 1$ jobs, $dp[i - 1]$.
- **Take it:** gain $v[i]$, then the best among jobs compatible with i , which is exactly $dp[p[i]]$.

The Recurrence

For job i :

- **Skip it:** the best among the first $i - 1$ jobs, $dp[i - 1]$.
- **Take it:** gain $v[i]$, then the best among jobs compatible with i , which is exactly $dp[p[i]]$.

$$dp[i] = \max(dp[i - 1], v[i] + dp[p[i]])$$

Base case $dp[0] = 0$. Fill $dp[1], \dots, dp[n]$ in order.

$p[i]$ is what makes this work: it skips over every incompatible job at once.

Working the Example

i	0	1	2	3	4	5
$v[i]$		2	4	3	5	7
$p[i]$		0	0	1	3	0
$dp[i]$	0	2	4	5	10	10

- $dp[3] = \max(dp[2], 3 + dp[1]) = \max(4, 5) = 5$.
- $dp[4] = \max(dp[3], 5 + dp[3]) = \max(5, 10) = 10$.
- $dp[5] = \max(dp[4], 7 + dp[0]) = \max(10, 7) = 10$.

Answer 10, from jobs $\{1, 3, 4\}$.

Implementation: Preprocessing

Sort by finish time, then find each $p[i]$ by binary search.

```
function weighted_interval_scheduling(starts, finishes, weights)
    n = length(starts)

    order = sortperm(finishes) # sort by finish time
    s, f, v = starts[order], finishes[order], weights[order]

    p = zeros{Int, n} # p[i] by binary search
    for i in 1:n
        lo, hi = 1, i - 1
        while lo <= hi
            mid = div(lo + hi, 2)
            if f[mid] <= s[i]
                p[i] = mid; lo = mid + 1
            else
                hi = mid - 1
            end
        end
    end

    # continued on the next slide
end
```

Implementation: Table and Traceback

```
# ... still inside weighted_interval_scheduling

dp = OffsetArray(zeros{Int, n + 1}, 0:n)
for i in 1:n
    dp[i] = max(dp[i - 1], v[i] + dp[p[i]])
end

selected = Int[] # trace back
i = n
while i >= 1
    if v[i] + dp[p[i]] > dp[i - 1]
        push!(selected, order[i]) # job i was taken
        i = p[i] # jump over conflicts
    else
        i -= 1
    end
end

return dp[n], sort(selected)
end
```

Running Time and Traceback

- Sorting: $O(n \log n)$.
- Computing every $p[i]$ by binary search: $O(n \log n)$.
- Filling the table: $O(n)$.

$$O(n \log n)$$

To recover the jobs, walk back from $i = n$: if $v[i] + dp[p[i]] > dp[i - 1]$, job i was taken, so record it and jump to $p[i]$; otherwise move to $i - 1$.